THEORY OF CONCURRENCY

Congruence and abstraction

From strong to weak and branching bisimulation

Lecture 5 Piotr Hofman

First: when does equivalence support substitution?

Then: how much internal behaviour may an abstraction forget?

Congruence: equality compatible with operations

Let $\mathcal{A}$ be a set equipped with operations $f : \mathcal{A}^n \to \mathcal{A}$.

Definition

An equivalence relation $\equiv$ is a **congruence** if, for every operation f,

$$x_i \equiv y_i \ (1 \le i \le n) \quad \Longrightarrow \quad f(x_1, \ldots, x_n) \equiv f(y_1, \ldots, y_n).$$

A familiar example: integers modulo 2

If $x \equiv_2 x'$ and $y \equiv_2 y'$, then

$$x + y \equiv_2 x' + y', \qquad xy \equiv_2 x'y'.$$

Parity is a congruence for addition and multiplication.

The requirement depends on the chosen operations, not only on the equivalence relation.

The general algebraic definition; process congruence is its application to system constructors.

Why congruence gives a well-defined quotient

Every equivalence partitions $\mathcal{A}$ into classes $[x]$. We would like to define operations on classes by

$$\overline{f}([x_1], \ldots, [x_n]) = [f(x_1, \ldots, x_n)].$$

The role of congruence

The result must not depend on the representatives $x_1, \ldots, x_n$. This is *exactly* the compatibility requirement in the definition.

For systems

Build a larger system from the behavioural classes of its components, without inspecting which representatives were chosen.

Congruence makes "compute with behaviours instead of implementations" mathematically meaningful.

Processes and a small collection of operations

For now: every action is visible

An LTS is $T = (S, \Sigma, \to)$, with $\to \subseteq S \times \Sigma \times S$. A process is an LTS with a distinguished state.

$\mathbf{0}$	No outgoing transitions.
$a.P$	Perform $a \in \Sigma$, then behave as P.
$P + Q$	Perform a first transition of one summand; discard the other.
$\mathrm{block}_H(P)$	Disable transitions labelled in $H \subseteq \Sigma$.

These are operations on LTSs; expressions are just convenient notation.

No acceptance predicate or successful-termination marker. Cyclic processes may be given directly as graphs.

The operations are defined by transition rules

Prefix and choice

$$\frac{}{a.P \xrightarrow{a} P} \qquad \frac{P \xrightarrow{a} P'}{P + Q \xrightarrow{a} P'} \qquad \frac{Q \xrightarrow{a} Q'}{P + Q \xrightarrow{a} Q'}.$$

Blocking: restriction to allowed actions

$$\frac{P \xrightarrow{a} P'}{\mathrm{block}_H(P) \xrightarrow{a} \mathrm{block}_H(P')} \qquad a \notin H.$$

No rule permits an action in H.

$$a.b.\mathbf{0} + a.c.\mathbf{0} \xrightarrow{a} b.\mathbf{0}, \qquad \mathrm{block}_{\{c\}}(c.\mathbf{0}) \sim \mathbf{0}.$$

Prefix and choice follow the supplied CCS rules; blocking is the label-based restriction operation.

Process congruence: replacement in every context

A finite **one-hole context** is built from the chosen operations:

$$C[-] ::= [-] \mid a.C[-] \mid C[-] + R \mid R + C[-] \mid \mathrm{block}_H(C[-]).$$

Here R is any fixed process. $C[P]$ is obtained by filling the hole with P.

Equivalent formulation of compatibility

$$P \equiv Q \quad \Longrightarrow \quad C[P] \equiv C[Q] \quad \text{for every allowed } C[-].$$

A different requirement from logical preservation

Satisfying the same formulas concerns *observations*. Congruence concerns *operations that build larger systems*.

Adapted from the supplied context definitions; see also [?].

Congruence: from basic operations to contexts

Lemma

Let $\equiv$ be an equivalence relation on processes. The following conditions are equivalent:

1. $\equiv$ is compatible with every basic process operation.
2. For every finite one-hole context $C[-]$ built from these operations and fixed processes,

$$P \equiv Q \quad \Longrightarrow \quad C[P] \equiv C[Q].$$

Proof sketch

$(1) \Rightarrow (2)$**.** Structural induction on $C[-]$: apply compatibility at each constructor and reflexivity for the fixed arguments.

$(2) \Rightarrow (1)$**.** Fix all arguments of a basic operation except one. This gives a one-hole context, hence compatibility in that argument. Replace the arguments one at a time and use transitivity.

Strong bisimilarity is a congruence

Theorem for the operators just defined

$$P \sim Q \quad \Longrightarrow \quad C[P] \sim C[Q].$$

Example: preservation by choice. Assume $P \sim Q$ and fix R.

- A move of $P + R$ coming from P is matched by a move of Q with a bisimilar successor.
- A move coming from R is matched by the identical move of R.

The same reasoning applies in the reverse direction.

Compatibility for the displayed constructors; see [?].

Trace equivalence is also a congruence

Use the language of **all finite traces**, including the empty word:

$$\mathrm{Tr}(P) = \{w \in \Sigma^* : P \xrightarrow{w} P' \text{ for some } P'\}.$$

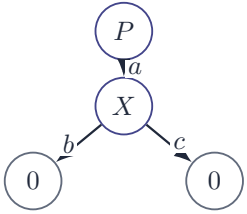
The trace language of a composite is determined by its operands

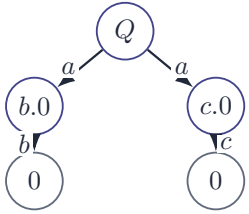
$$\begin{aligned}
\mathrm{Tr}(\mathbf{0}) &= \{\varepsilon\},\\
\mathrm{Tr}(a.P) &= \{\varepsilon\} \cup a\,\mathrm{Tr}(P),\\
\mathrm{Tr}(P+Q) &= \mathrm{Tr}(P) \cup \mathrm{Tr}(Q),\\
\mathrm{Tr}(\mathrm{block}_H(P)) &= \mathrm{Tr}(P) \cap (\Sigma \setminus H)^*.
\end{aligned}$$

Congruence *alone* does not justify choosing bisimulation over trace equivalence. We also need to specify what must be observed.

The same traces, different choices after interaction

$$P = a.(b.\mathbf{0} + c.\mathbf{0}), \qquad Q = a.b.\mathbf{0} + a.c.\mathbf{0}.$$





$$\mathrm{Tr}(P) = \mathrm{Tr}(Q) = \{\varepsilon, a, ab, ac\}, \qquad P \not\sim Q.$$

P satisfies $\langle a\rangle(\langle b\rangle\top \wedge \langle c\rangle\top)$; Q does not.

Deadlock observations can destroy congruence

A **finite completed trace** ends in a deadlock:

$$\mathrm{CT}(P) = \{w : P \xrightarrow{w} P' \text{ and } P' \text{ has no outgoing transition}\}.$$

For the preceding P and Q, $\mathrm{CT}(P) = \mathrm{CT}(Q) = \{ab, ac\}$.

An environment that blocks c

$$\mathrm{CT}(\mathrm{block}_{\{c\}}(P)) = \{ab\},$$
$$\mathrm{CT}(\mathrm{block}_{\{c\}}(Q)) = \{a, ab\}.$$

The second process can commit to its c-branch and become stuck after a.

The ordinary trace languages still agree. Completed-trace equality is not a congruence for blocking.

Why make bisimulation our baseline?

The verification objective

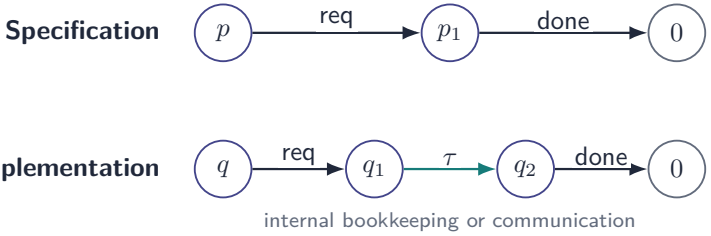
Preserve the available continuations after every matched interaction, including branching observations and deadlock behaviour.

Logical	Strong bisimulation preserves HML; on Kripke structures, its label-preserving version preserves CTL*.
	Interaction can not be defined on the basis of traces.
Algebraic	It supports substitution.
Proof-theoretic	A witness is checked locally by matching transitions; finite-state minimisation uses partition refinement.

Now add internal actions and abstraction

Let $\Sigma_\tau = \Sigma \cup \{\tau\}$, with $\tau \notin \Sigma$.

$a, b, c \in \Sigma$ are visible; $\alpha \in \Sigma_\tau$ may be internal.



External behaviour

Strong bisimulation distinguishes the initial states because of the internal step. We seek a coarser relation that can abstract from this bookkeeping.

The simplified process algebra with internal actions

Let Σ contain the visible actions, with $\tau \notin \Sigma$. Write

$$\Sigma_\tau = \Sigma \cup \{\tau\}, \qquad \alpha \in \Sigma_\tau, \qquad H \subseteq \Sigma.$$

Process constructors

$\mathbf{0}$	No outgoing transitions, not even internal ones.
$\alpha.P$	Perform α, then behave as P. In particular, $\tau.P \xrightarrow{\tau} P$.
$P + Q$	The first transition of either summand selects that summand and discards the other, even if its label is τ.
$\mathrm{block}_H(P)$	Remove transitions labelled in H. Other transitions, including τ-transitions, remain.
$\mathrm{hide}_H(P)$ (new)	Replace labels in H by τ. Retain all states and transitions; leave other labels unchanged.

The simplified process algebra with internal actions

Let Σ contain the visible actions, with $\tau \notin \Sigma$. Write

$$\Sigma_\tau = \Sigma \cup \{\tau\}, \qquad \alpha \in \Sigma_\tau, \qquad H \subseteq \Sigma.$$

Blocking prevents an action; hiding conceals it

$$\mathrm{block}_{\{a\}}(a.b.\mathbf{0}) \sim \mathbf{0}, \qquad \mathrm{hide}_{\{a\}}(a.b.\mathbf{0}) \sim \tau.b.\mathbf{0}.$$

After hiding a, the process can still reach and perform b.

Operational rules: an internal step is still a step

Throughout, $\alpha \in \Sigma_\tau$ and $H \subseteq \Sigma$. The process $\mathbf{0}$ has no transition rules.

Prefix and choice: the same rules for visible and internal actions

$$\frac{}{\alpha.P \xrightarrow{\alpha} P} \qquad \frac{P \xrightarrow{\alpha} P'}{P + Q \xrightarrow{\alpha} P'} \qquad \frac{Q \xrightarrow{\alpha} Q'}{P + Q \xrightarrow{\alpha} Q'}.$$

Blocking and hiding

$$\frac{P \xrightarrow{\alpha} P'}{\mathrm{block}_H(P) \xrightarrow{\alpha} \mathrm{block}_H(P')}\ (\alpha \notin H) \qquad \frac{P \xrightarrow{\alpha} P'}{\mathrm{hide}_H(P) \xrightarrow{h_H(\alpha)} \mathrm{hide}_H(P')},$$

where

$$h_H(\alpha) = \begin{cases} \tau, & \alpha \in H, \\ \alpha, & \alpha \notin H. \end{cases}$$

In particular, $h_H(\tau) = \tau$.

Operational rules: an internal step is still a step

A silent transition can resolve a choice

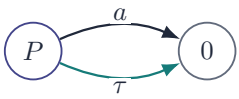
$$\tau.a.\mathbf{0} + b.\mathbf{0} \xrightarrow{\tau} a.\mathbf{0}, \qquad \tau.a.\mathbf{0} + b.\mathbf{0} \xrightarrow{b} \mathbf{0}.$$

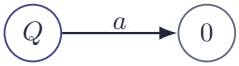
Both initial transitions are possible: τ has no priority. If the τ-step occurs, the successor is $a.\mathbf{0}$, **not** $a.\mathbf{0} + b.\mathbf{0}$: the b-alternative disappears.

A tempting but inadequate proposal: ignore silent moves

Suppose we compare only visible behaviour, allowing internal work before and after it, but impose *no obligation* on a purely internal move.

$$P = a.\mathbf{0} + \tau.\mathbf{0}, \qquad Q = a.\mathbf{0}.$$





The silent-deadlock problem

Both can perform a, but only P can silently become stuck *before* a. The naive proposal ignores that commitment.

Internal transitions must also be matched, even when the matching response may do nothing.

Weak transitions

Zero or more internal steps

$$p \Rightarrow q \quad \Longleftrightarrow \quad p \xrightarrow{\tau*} q.$$

In particular, $p \Rightarrow p$ always holds.

One visible action, with internal work around it

For $a \in \Sigma$,

$$p \stackrel{a}{\Rightarrow} q \quad \Longleftrightarrow \quad \exists u, v : \; p \Rightarrow u \xrightarrow{a} v \Rightarrow q.$$

For uniform notation, define $p \stackrel{\tau}{\Rightarrow} q \iff p \Rightarrow q$.

A weak visible move contains exactly one visible action. A weak τ-move may be empty. All matching paths are finite.

Weak-bisimulation notation adapted from the supplied slides; see also [?].

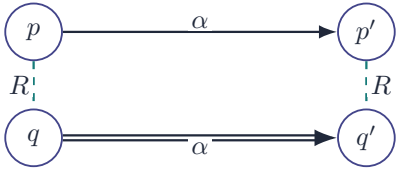
Weak bisimulation

Definition

A **symmetric** relation $R \subseteq S \times S$ is a weak bisimulation if

$$p \, R \, q \;\wedge\; p \xrightarrow{\alpha} p' \quad\Longrightarrow\quad \exists q' :\; q \overset{\alpha}{\Longrightarrow} q' \;\wedge\; p' \, R \, q' \qquad (\alpha \in \Sigma_\tau).$$

We write $p \approx_{\mathrm{w}} q$ if some weak bisimulation relates p and q.



Symmetry supplies the same obligation for moves from q.

Weak bisimilarity is the greatest weak bisimulation and is an equivalence relation.

Weak Hennessy–Milner logic

Recall

$$p \Rightarrow q \quad \Longleftrightarrow \quad p \xrightarrow{\tau^*} q,$$

and, for $a \in \Sigma$,

$$p \stackrel{a}{\Rightarrow} q \quad \Longleftrightarrow \quad p \xrightarrow{\tau^*} \xrightarrow{a} \xrightarrow{\tau^*} q.$$

Weak modalities

$$\varphi ::= \top \mid \neg\varphi \mid \varphi \wedge \psi \mid \langle\!\langle \alpha \rangle\!\rangle \varphi, \qquad \alpha \in \Sigma \cup \{\tau\}.$$

$$p \models \langle\!\langle \alpha \rangle\!\rangle \varphi \quad \Longleftrightarrow \quad \exists q :\ p \stackrel{\alpha}{\Rightarrow} q \wedge q \models \varphi.$$

For $\alpha = \tau$, we put $\stackrel{\tau}{\Rightarrow} = \Rightarrow$, so zero or more internal steps are allowed.

The logic observes the endpoint of a weak transition, but not the states crossed by its internal part.

Weak HML characterizes weak bisimulation

Theorem

For finite LTSs,

$$p \approx_{\mathrm{w}} q \quad \iff \quad \forall \varphi \in \mathrm{WHML} : (p \models \varphi \iff q \models \varphi).$$

Why? Saturate the transition system

Construct $\mathrm{Sat}(T)$ by putting

$$p \xrightarrow{a}_{\mathrm{w}} q \quad \iff \quad p \stackrel{a}{\Rightarrow} q, \text{ and}$$

$$p \xrightarrow{\tau}_{\mathrm{w}} q \quad \iff \quad p \Rightarrow q.$$

Then

$$p \approx_{\mathrm{w}} q \text{ in } T \quad \iff \quad p \sim q \text{ in } \mathrm{Sat}(T).$$

Apply the ordinary Hennessy–Milner theorem to $\mathrm{Sat}(T)$.

What if the logic can inspect a silent computation?

Weak HML sees only

$$p \stackrel{a}{\Rightarrow} q.$$

It cannot ask what remains possible *while* the silent preparation for a is taking place.

A CTL-style "until" operator

Introduce

$$\varphi \langle a \rangle \psi$$

with the semantics

$$p \models \varphi \langle a \rangle \psi$$

iff there exist

$$p = p_0 \xrightarrow{\tau} p_1 \xrightarrow{\tau} \cdots \xrightarrow{\tau} p_n \xrightarrow{a} q$$

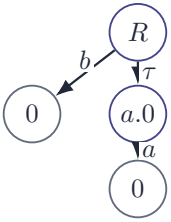
such that

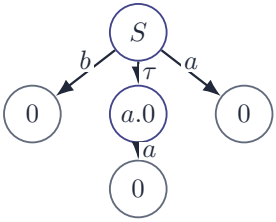
$$p_i \models \varphi \quad (0 \le i \le n), \qquad q \models \psi.$$

Until can distinguish weakly bisimilar processes

Consider

$$R = \tau.a.\mathbf{0} + b.\mathbf{0}, \qquad S = a.\mathbf{0} + \tau.a.\mathbf{0} + b.\mathbf{0}.$$





$$R \approx_{\mathrm{w}} S.$$

But let

$$\chi = (\langle\langle b \rangle\rangle \top) \langle a \rangle \top.$$

Then

$$S \models \chi, \qquad R \not\models \chi.$$

What exactly does the formula observe?

$$\chi = (\langle\langle b \rangle\rangle \top)\langle a \rangle \top.$$

$S \models \chi$

S may perform a immediately:

$$S \xrightarrow{a} 0.$$

Before a happens, we are still in S, where b is possible.

$R \not\models \chi$

Every execution of a has to start with

$$R \xrightarrow{\tau} a.0 \xrightarrow{a} 0.$$

After the silent step the b-alternative has disappeared:

$$a.0 \not\models \langle\langle b \rangle\rangle \top.$$

Branching bisimulation

Definition

A symmetric relation R is a **branching bisimulation** if $p \, R \, q$ and

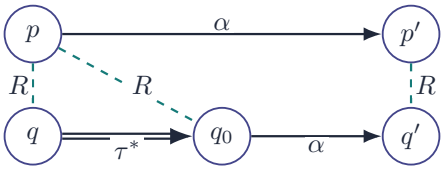
$$p \xrightarrow{\alpha} p'$$

imply either

$$\alpha = \tau \quad \text{and} \quad p' \, R \, q,$$

or there exist q_0, q' such that

$$q \Rightarrow q_0 \xrightarrow{\alpha} q', \qquad p \, R \, q_0, \qquad p' \, R \, q'.$$



Weak versus branching bisimulation

For the previous processes,

$$R = \tau.a.0 + b.0, \qquad S = a.0 + \tau.a.0 + b.0,$$

we have

$$R \approx_{\mathrm{w}} S, \qquad R \not\approx_{\mathrm{b}} S.$$

Why does branching bisimulation fail?

Consider the direct transition

$$S \xrightarrow{a} 0.$$

The only way R can prepare an a is

$$R \xrightarrow{\tau} a.0 \xrightarrow{a} 0.$$

Branching bisimulation would require

$$S \approx_{\mathrm{b}} a.0$$

Hennessy–Milner logic with Until

HMLU

$$\varphi ::= \top \mid \neg\varphi \mid \varphi \wedge \psi \mid \varphi\langle\alpha\rangle\psi.$$

The last operator requires an internal path

$$p = p_0 \xrightarrow{\tau} \cdots \xrightarrow{\tau} p_n \xrightarrow{\alpha} q$$

such that every p_i satisfies φ and q satisfies ψ.

Theorem

For finite LTSs,

$$p \approx_{\mathrm{b}} q \iff p \text{ and } q \text{ satisfy the same HMLU formulas.}$$

$$\underbrace{\text{weak HML}}_{\text{endpoint only}} \rightsquigarrow \approx_{\mathrm{w}} \qquad \underbrace{\text{HMLU}}_{\text{inspect silent prefix}} \rightsquigarrow \approx_{\mathrm{b}}.$$

Infinite silence is a separate question

Consider

$$p \xrightarrow{a} 0 \qquad q \xrightarrow{a} 0, \qquad q \xrightarrow{\tau} q.$$

Ordinary weak and branching bisimulation

$$p \approx_{\mathrm{w}} q, \qquad p \approx_{\mathrm{b}} q.$$

The τ-loop can be ignored forever.

But there is an observable design question

q admits an infinite internal computation

$$q \xrightarrow{\tau} q \xrightarrow{\tau} q \xrightarrow{\tau} \cdots,$$

while p does not.

If this distinction matters, use a **divergence-preserving** version of the equivalence (and a corresponding divergence-sensitive logic).

Weak and branching bisimilarity are not congruences

Both equivalences abstract from an initial finite silent computation:

$$\tau.0 \approx_{\mathrm{w}} 0, \qquad \tau.0 \approx_{\mathrm{b}} 0.$$

Now put the processes in the choice context

$$C[-] = a.0 + [-].$$

Then

$$C[\tau.0] = a.0 + \tau.0, \qquad C[0] = a.0 + 0.$$

But

$$a.0 + \tau.0 \not\approx_{\mathrm{w}} a.0 + 0.$$

The left process can silently choose the second branch and deadlock, while the right process cannot.
Hence also

$$a.0 + \tau.0 \not\approx_{\mathrm{b}} a.0 + 0.$$

The problem is at the root: an initial silent step may resolve an external choice.

Rooted weak bisimilarity

To recover substitution under choice, treat the first move more strictly.
Define

$$q \stackrel{\widehat{\alpha}}{\Longrightarrow} q' = \begin{cases} q \Rightarrow \stackrel{a}{\rightarrow} \Rightarrow q', & \alpha = a \in \Sigma, \\ q \Rightarrow \stackrel{\tau}{\rightarrow} \Rightarrow q', & \alpha = \tau. \end{cases}$$

Notice that a root τ must now be matched by **at least one** τ.

Rooted weak bisimilarity

$$p \approx_{\mathrm{rw}} q$$

iff every initial

$$p \stackrel{\alpha}{\rightarrow} p'$$

can be matched by

$$q \stackrel{\widehat{\alpha}}{\Longrightarrow} q' \qquad \text{with} \qquad p' \approx_{\mathrm{w}} q',$$

and symmetrically.

Rooted branching bisimilarity

For branching bisimulation the standard root condition is even cleaner.

Definition

$$p \approx_{\mathrm{rb}} q$$

iff every initial transition

$$p \xrightarrow{\alpha} p'$$

can be matched by an actual transition

$$q \xrightarrow{\alpha} q'$$

such that

$$p' \approx_{\mathrm{b}} q',$$

and symmetrically.

Compositional role

The rooted variants repair the failure under nondeterministic choice. For the standard CCS-style operators they give the corresponding congruences.

Rootedness and branching preservation solve different problems: branching controls what may happen *inside* a silent computation; rootedness controls how a component may be used by its environment.

What τ adds to the picture

Observation	Logic	Equivalence
every transition	HML	strong bisimulation
endpoints of silent computations	weak HML	weak bisimulation
branching preserved during silence	HML with Until	branching bisimulation
finite stuttering in branching time	CTL$^*-X$	branching bisimulation

Two additional refinements answer independent questions:

divergence preservation and rootedness / congruence.

There is no single way to "ignore τ": the equivalence depends on what the observer is allowed to see.

Application: extend the language with communication

Visible actions come in complementary pairs $a, \bar{a}$, with $\overline{\bar{a}} = a$. Only complementary visible actions synchronise.

A component can move on its own

$$\frac{P \xrightarrow{\alpha} P'}{P \parallel Q \xrightarrow{\alpha} P' \parallel Q} \qquad \frac{Q \xrightarrow{\alpha} Q'}{P \parallel Q \xrightarrow{\alpha} P \parallel Q'}.$$

Two components can communicate

$$\frac{P \xrightarrow{a} P' \qquad Q \xrightarrow{\bar{a}} Q'}{P \parallel Q \xrightarrow{\tau} P' \parallel Q'}.$$

Synchronisation is possible, but not compulsory. Blocking both a and $\bar{a}$ forces a private handshake.

Add parallel contexts $C[-] \parallel R$ and $R \parallel C[-]$. Strong and rooted congruence extend to them [?].

An internal handshake gives a replaceable implementation

$$Client = \mathsf{req}.\bar{h}.\mathsf{done}.\mathbf{0}, \qquad Server = h.\mathbf{0},$$
$$System = \mathrm{block}_{\{h,\bar{h}\}}(Client \parallel Server), \qquad Spec = \mathsf{req}.\mathsf{done}.\mathbf{0}.$$



What blocking enforces

The h and $\bar{h}$ actions cannot occur separately. Their joint transition is labelled τ and is not blocked.

$System \approx_{\mathrm{rb}} Spec$. Both start with req; their derivatives are branching bisimilar.

Confluence: determinism of the result

A system may be locally nondeterministic:

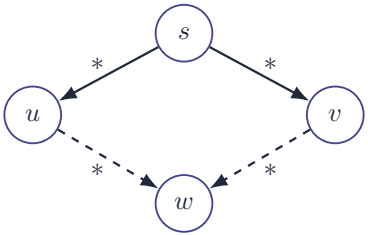
$$s \xrightarrow{*} u, \qquad s \xrightarrow{*} v.$$

The two computations may make different choices.

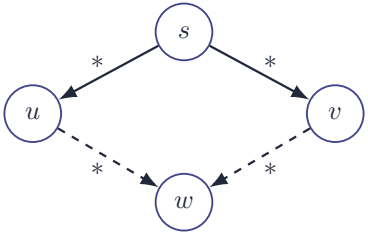
Confluence

A reachability relation $\to$ is **confluent** if

$$s \xrightarrow{*} u \ \wedge \ s \xrightarrow{*} v \quad \Longrightarrow \quad \exists w : \ u \xrightarrow{*} w \ \wedge \ v \xrightarrow{*} w.$$



Confluence: determinism of the result



Confluence is not determinism of the next step: different choices are allowed, but they cannot create an irreversible disagreement.

Confluence and unique outcomes

Confluence alone does *not* say that every computation terminates.

$$s \xrightarrow{*} s \xrightarrow{*} s \xrightarrow{*} \cdots$$

may be perfectly confluent.

If termination is added

If $\to$ is

- terminating, and
- confluent,

then every state has at most one normal form.

Thus different reduction strategies may pass through different states, but if they terminate, they terminate in the same result:

$$s \xrightarrow{*} n_1, \qquad s \xrightarrow{*} n_2, \qquad n_1, n_2 \text{ normal} \quad \Longrightarrow \quad n_1 = n_2.$$

Confluence and unique outcomes

Thus different reduction strategies may pass through different states, but if they terminate, they terminate in the same result:

$$s \xrightarrow{*} n_1, \qquad s \xrightarrow{*} n_2, \qquad n_1, n_2 \text{ normal} \implies n_1 = n_2.$$

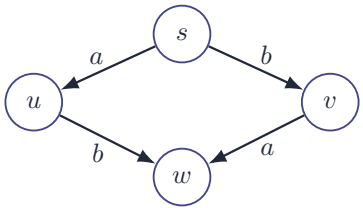
A useful intuition is: **confluence = determinism modulo temporary choices**.

From confluence to commuting computations

In concurrent systems the same phenomenon often appears locally.
Suppose

$$s \xrightarrow{a} u, \qquad s \xrightarrow{b} v.$$

If the two actions do not interfere, we may obtain a commuting square:



The interleavings

$$ab \qquad \text{and} \qquad ba$$

differ as words, but may represent the same concurrent computation.

Important distinction

From confluence to commuting computations

The interleavings

$$ab \qquad \text{and} \qquad ba$$

differ as words, but may represent the same concurrent computation.

Important distinction

Confluence says that computations can be joined. It does *not* by itself tell us which actions are independent.

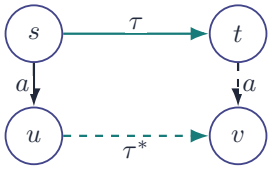
Commuting diamonds suggest that some ordering information in an interleaving trace may be accidental.

What should a harmless internal step do?

Suppose

$$s \xrightarrow{\tau} t \qquad \text{and} \qquad s \xrightarrow{a} u.$$

If the internal step is behaviourally harmless, performing it first should not destroy the visible possibility a.
Ideally the two orders can be reconciled:



Interpretation

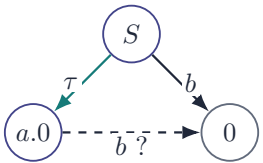
Doing the internal work first, or doing the visible action first, leads to behaviourally compatible states.

Confluence of τ-steps alone is not enough

Consider

$$S = \tau.a.\mathbf{0} + b.\mathbf{0}.$$

The relation consisting only of τ-steps is trivially confluent: there is only one internal transition.



But the τ-step is not harmless

Before the internal step, b is possible.

After

$$S \xrightarrow{\tau} a.0,$$

the b-alternative has disappeared.

$$S \not\approx_{\mathrm{b}} a.0.$$

For behavioural reduction, an internal transition must commute with *all competing behaviour*, not only with other internal transitions.

A local criterion: T-confluence

Choose a set T of internal transitions that we would like to treat as harmless.

Idea

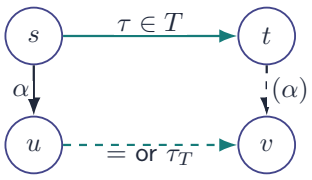
Whenever a T-transition competes with another transition, performing either one first should lead to the same state, possibly after one further T-step.

For every

$$s \xrightarrow{\tau}_T t \qquad \text{and} \qquad s \xrightarrow{\alpha} u,$$

require some v such that

$$t \xrightarrow{(\alpha)} v, \qquad u = v \quad \text{or} \quad u \xrightarrow{\tau}_T v.$$



A T-step can be moved before a competing action without changing the resulting observable behaviour.

[?, Section 2].

Confluent internal transitions are inert

Theorem

If the LTS is T-confluent, then

$$s \xrightarrow{\tau}_T t \quad \Longrightarrow \quad s \approx_{\mathrm{b}} t.$$

Such a transition is called **inert**: crossing it does not change the branching behaviour of the process.

Why?

Every transition competing with

$$s \xrightarrow{\tau}_T t$$

can be moved across the T-transition using the confluence square. Hence the states before and after the internal transition can match each other's observable moves.

local commutation $\Longrightarrow$ behavioural equivalence

From inertness to state-space reduction

If an internal transition is confluent, it is tempting to perform it immediately and ignore the competing transitions.



But one more condition is needed

If internal transitions can continue forever, always giving them priority may prevent observable behaviour forever.

Therefore, under T-confluence and suitable τ-convergence, confluent internal transitions can be prioritised soundly.

Confluence explains why an alternative is redundant; convergence ensures that prioritising internal work does not trap the execution forever.

Why convergence still matters



The internal transition is inert, but naive priority is unsound

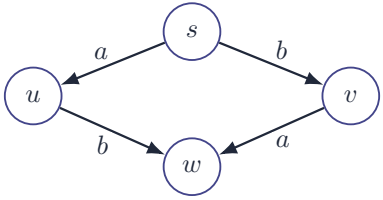
The τ-loop does not change the state, so it is trivially confluent.
But if the scheduler always chooses it, the visible action a disappears from the reduced system.

inert transition $\not\Rightarrow$ arbitrary deletion of competing transitions.

Confluence deals with finite reordering. Divergence is a separate issue.

From confluence to non-interleaving semantics

Confluence has repeatedly produced diagrams of the form



Interleaving semantics records two executions:

$$ab \qquad \text{and} \qquad ba.$$

But if a and b are independent, these words differ only by an arbitrary scheduling decision.

Next step

Mazurkiewicz traces identify words that differ only by exchanging adjacent independent actions:

$$uabv \equiv_I ubav \qquad \text{whenever } (a, b) \in I.$$

What did internal actions force us to distinguish?

Observation	Traces record executions; bisimulation also records available continuations.
Finite silence	Weak bisimulation hides finite internal work; branching bisimulation additionally preserves branching during it.
Infinite silence	Divergence-sensitive variants distinguish infinite internal computation from finite stuttering.
Composition	Rooted refinements restore substitution in contexts such as nondeterministic choice.
Confluence	Some nondeterministic choices commute and can be treated as behaviourally irrelevant.

Optional material: weak bisimulation game and saturation.

OPTIONAL

The weak bisimulation game

A position is a pair of states (p, q).

1. Spoiler chooses either side and one transition, say $p \xrightarrow{\alpha} p'$.
2. Duplicator chooses a *finite* matching path $q \stackrel{\alpha}{\Longrightarrow} q'$.
3. Continue from (p', q').

Winning condition

Spoiler wins when a chosen transition has no matching response.
Duplicator wins infinite plays, and positions where Spoiler has no move.

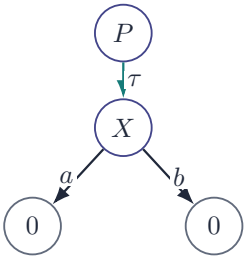
$p \approx_{\mathrm{w}} q$ exactly when Duplicator has a winning strategy from (p, q).

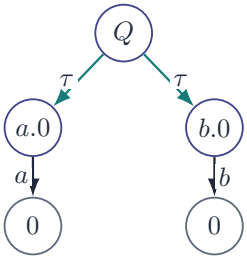
A silent commitment gives Spoiler a strategy

For $P = \tau.(a.\mathbf{0} + b.\mathbf{0})$ and $Q = \tau.a.\mathbf{0} + \tau.b.\mathbf{0}$, start with $Q \xrightarrow{\tau} a.\mathbf{0}$. Either response from P still permits b.

Invisible does not mean irrelevant

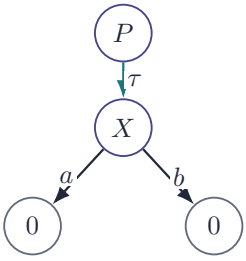
$$P = \tau.(a.\mathbf{0} + b.\mathbf{0}), \qquad Q = \tau.a.\mathbf{0} + \tau.b.\mathbf{0}.$$

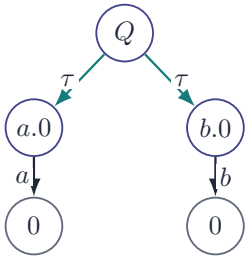




Invisible does not mean irrelevant

$$P = \tau.(a.\mathbf{0} + b.\mathbf{0}), \qquad Q = \tau.a.\mathbf{0} + \tau.b.\mathbf{0}.$$





$P \not\approx_{\mathrm{w}} Q$: after $Q \xrightarrow{\tau} a.0$, every possible silent response of P still allows b.

OPTIONAL

Weak bisimilarity by saturation

Saturated LTS

Keep the states and add an edge for every weak transition:

$$p \xrightarrow{\alpha}_{\mathrm{sat}} q \quad \Longleftrightarrow \quad p \overset{\alpha}{\Longrightarrow} q \qquad (\alpha \in \Sigma_\tau).$$

This includes *all* τ^*-shortcuts, not just τ-self-loops.

Characterisation

$$p \approx_{\mathrm{w}} q \text{ in } T \quad \Longleftrightarrow \quad p \sim q \text{ in } T_{\mathrm{sat}}.$$

Compute τ-reachability, build the saturated graph, then reuse strong-bisimulation partition refinement.

Adapted and made explicit from the saturation idea in the supplied weak-bisimulation slides.

OPTIONAL

Why saturation works, and what it costs

From weak to strong. Repeatedly apply the weak transfer condition along the finite path

$$\tau^* \, a \, \tau^* \quad \text{or} \quad \tau^*.$$

Concatenating the responses gives a matching saturated edge with a related endpoint.

From strong to weak. Every original edge is a saturated edge; a matching saturated edge is exactly an allowed weak response.

Finite-state cost

With n states and $k = |\Sigma_\tau|$ labels, the saturated LTS can have up to kn^2 edges. The reduction can destroy sparsity. Include the cost of constructing saturation, not only of minimising it.

This also proves that weak bisimilarity is an equivalence relation.

OPTIONAL

The precise divergence-preservation clause

Let R be a symmetric branching bisimulation.

Explicit divergence condition

If $p \, R \, q$ and there is an infinite path

$$p = p_0 \xrightarrow{\tau} p_1 \xrightarrow{\tau} p_2 \cdots \quad \text{with } p_i \, R \, q \text{ for every } i,$$

then there must be an infinite path

$$q = q_0 \xrightarrow{\tau} q_1 \xrightarrow{\tau} q_2 \cdots \quad \text{with } p_i \, R \, q_j \text{ for all } i, j.$$

We write $p \approx_{\mathrm{b}}^{\Delta} q$ if some branching bisimulation satisfying this condition relates them.

The condition concerns divergence *within related behaviour*; it is not merely an extra Boolean label saying "can diverge".

[?, Section 3]; also [?, Definition 2.1].

OPTIONAL

The inertness proof in detail

Let $s \xrightarrow{\tau}_T t$ and put $R = \mathrm{Id} \cup T \cup T^{-1}$.

A move from s. For $s \xrightarrow{\alpha} u$, confluence gives

$$t \xrightarrow{(\alpha)} v, \qquad u = v \text{ or } u \xrightarrow{\tau}_T v.$$

The source pair is $(s, t) \in R$ and the target pair is $(u, v) \in R$.

A move from t. For $t \xrightarrow{\alpha} v$, respond with

$$s \xrightarrow{\tau}_T t \xrightarrow{\alpha} v.$$

Use the identity pairs (t, t) before the matching action and (v, v) afterwards.

Identity and converse pairs give the remaining cases. Thus R is a branching bisimulation.

Proof for the sufficient condition in [?].

OPTIONAL

Weak modal observations

Replace the one-step modality of Hennessy–Milner logic by a weak modality:

$$\varphi ::= \top \mid \neg\varphi \mid \varphi \wedge \varphi \mid \langle\!\langle \alpha \rangle\!\rangle \varphi, \qquad \alpha \in \Sigma_\tau,$$

$$p \models \langle\!\langle \alpha \rangle\!\rangle \varphi \quad \Longleftrightarrow \quad \exists q : \; p \stackrel{\alpha}{\Longrightarrow} q \;\wedge\; q \models \varphi.$$

Finite LTSs

Two states are weakly bisimilar iff they satisfy the same formulas of this logic.

Reason: apply the Hennessy–Milner theorem to the saturated LTS.

Keep the τ-modality

$a.\mathbf{0} + \tau.\mathbf{0}$ satisfies $\langle\!\langle \tau \rangle\!\rangle \neg \langle\!\langle a \rangle\!\rangle \top$, but $a.\mathbf{0}$ does not.

References: abstraction and congruence

[GW96] R. J. van Glabbeek and W. P. Weijland. *Branching Time and Abstraction in Bisimulation Semantics*. Journal of the ACM 43(3), 555–600, 1996.

Author-hosted version.

[GLT09] R. J. van Glabbeek, B. Luttik, and N. Trčka. *Branching Bisimilarity with Explicit Divergence*. Fundamenta Informaticae 93(4), 371–392, 2009.

arXiv:0812.3068.

[GLS20] R. J. van Glabbeek, B. Luttik, and L. Spaninks. *Rooted Divergence-Preserving Branching Bisimilarity is a Congruence*. Logical Methods in Computer Science 16(3:14), 2020.

arXiv:1801.01180.

References: reduction and source material

[GP00] J. F. Groote and J. van de Pol. *State Space Reduction using Partial τ-Confluence*. CWI report SEN-R0008, 2000, especially Sections 2 and 5.

CWI repository record.

[MCS] R. J. van Glabbeek. *Modelling Concurrent Systems: course notes*. University of Edinburgh, 2024, especially Lectures 6–8.

Author's course notes.

Basis and additions

The supplied weak-bisimulation and process-algebra slides provide the starting material. Branching bisimulation, divergence, and rootedness are developed for this lecture. The former global CCS-confluence section is replaced by the local τ-reduction criterion above.

A source map, corrections to the old slides, and teaching cautions are recorded in `lecture5_notes.md`.